

CS 133 Lab 4 Report: FPGA Based Parallel CNN Acceleration Using HLS

Name: Alexander Neary

UID: 805935345

Date: May 2026

1. Parallelization and Optimization Strategies

- **Loop Pipelining:** I applied `#pragma HLS PIPELINE II=1` to the innermost loops (the convolution accumulation). This forces the HLS tool to initiate one operation per clock cycle, maximizing throughput by overlapping the execution of consecutive iterations.
 - **Loop Unrolling:** I used `#pragma HLS UNROLL` on the filter dimensions (5 x 5) and channel dimension (16). This converts the loop into hardware logic that executes in parallel rather than sequentially, which is essential to meet the computational demands of the 5 x 5 window.
 - **Vectorized Memory Access:** I implemented `hls::vector` types (e.g., `float4`, `float16`) to facilitate burst memory transfers. By reading/writing 16 floats at once (512 bits), I effectively utilized the AXI-MM interface bandwidth, reducing the number of total memory transactions required.
 - **Array Partitioning:** To enable the parallel processing of the unrolled loops, I applied `#pragma HLS ARRAY_PARTITION cyclic factor=...` on the input, weight, and output buffers. This creates multiple memory banks, preventing “bank conflicts” where the hardware tries to read or write to the same memory port simultaneously.
-

2. Incremental Evaluation

- **Baseline (Sequential):** The provided starter code, which achieved ~10 GFlops, served as the baseline.
 - **Optimization 1 (Pipelining and Unrolling):** Applying `PIPELINE` and `UNROLL` transformed the bottleneck from sequential clock cycles to parallel computation logic, resulting in the most significant GFlops jump.
 - **Optimization 2 (Array Partitioning):** This was the final step to resolve initiation interval (II) bottlenecks. Without partitioning, the loops could not achieve II=1 because of limited read/write ports on the BRAMs.
-

3. Difference from Lab 3 (CUDA)

- **Paradigm Shift:** In Lab 3 (CUDA), parallelism was achieved by launching thousands of threads across SMs. In Lab 4 (FPGA), there is no “thread” concept in the same way, instead I used spatial parallelism.
 - **Memory Strategy:** In CUDA, I relied on Shared Memory and coalescing global memory accesses. In this HLS design, I relied on BRAM-based local buffers and **ARRAY_PARTITION** to simulate high-bandwidth memory access for the hardware pipeline.
 - **Hardware Control:** The HLS approach allowed for much finer-grained control over the data path (via pragma-driven pipelining and unrolling) compared to the block-based thread model of CUDA
-

4. Resource Usage Analysis

The following results track the resource usage on the **m5.2xlarge** AWS instance.

Resource	Count	Percentage
BRAM_18K	4237	98%
DSP	2004	29%
FF	243,746	10%
LUT	153,401	12%
URAM	25	2%

Latency and GFlops: 1.048 seconds, 164.57 billion OPS / 1.048 seconds = 157 GFlops
Performance Range achieved: A+